3. TECHNICAL DOCUMENTATION

1. Introduction

What is RAG

Retrieval-Augmented Generation (RAG) combines:

- Information retrieval
- Language generation

It ensures responses are grounded in external knowledge.

Why Needed

LLMs alone:

- Hallucinate
- Lack domain context

RAG solves this by injecting relevant data.

---

2. System Architecture Explanation

The system integrates:

- Retrieval (ChromaDB)
- Generation (LLM)
- Workflow control (LangGraph)

LangGraph enables structured execution and decision-making.

---

3. Design Decisions

- **Chunk Size (400)**: Balances context vs precision
- **Overlap (50)**: Preserves continuity
- **Embeddings**: Chosen for semantic similarity
- **Retriever**: Top-k similarity search
- **Prompt Design**:
    - Inject retrieved context
    - Guide LLM to stay grounded

---

4. Workflow Explanation

- Query enters graph
- Processing node retrieves context
- LLM generates response
- Routing decides next step

---

5. Conditional Logic

- Based on:
    - Retrieval quality
    - Query complexity

Future improvement:

- Intent classification using LLM

---

6. HITL Implementation

Role

- Handles edge cases
- Improves reliability

Benefits

- Prevents wrong answers
- Builds trust

Limitations

- Slower response
- Requires human availability

---

7. Challenges & Trade-offs

- **Accuracy vs Speed**
- **Chunk Size vs Context Quality**
- **Cost vs Performance**

---

8. Testing Strategy

- Manual testing with queries
- Edge cases:
    - No context
    - Ambiguous queries
- Validation of responses

---

9. Future Enhancements

- Multi-document support
- Feedback loop
- Memory integration
- Web deployment
- Advanced routing (intent classifier)